FICHA DE IDENTIFICACIÓN DE TRABAJO DE INVESTIGACIÓN

Título	Optimización de redes neuronales discretas para Edge Computing en hardware comercial limitado: Caso Delizia.	
Autor/es	**Nombres y Apellidos**	**Código de estudiantes**
	Emilio Gabriel Fernández Jiménez	88078
Fecha	31/03/2026	

Carrera	Ing. De Sistemas
Asignatura	Redes Neuronales
Grupo	A
Docente	Ing. Roger Rolando Perez Rojas
Periodo Académico	I/2026
Subsede	Oruro

Copyright © (2026) por (Emilio Gabriel Fernández Jiménez). Todos los derechos reservados.

Tabla De Contenidos

Lista De Tablas 3
Lista De Figuras 4
Introducción 5
Capítulo 1. Planteamiento del Problema 7
1.1 Formulación del Problema 7
1.2 Objetivos 7
1.3 Justificación 8
1.4 Planteamiento de Hipótesis 8
Capítulo 2. Marco Teórico 9
2.1 Área de Estudio/Campo de Investigación 9
2.2 Desarrollo del Marco Teórico 9
2.2.1 Optimización de Redes Neuronales Discretas para Sistemas Embebidos y Edge Computing: Análisis Integral 9
2.2.2 Diferencias Arquitectónicas y Computacionales Frente a Redes Neuronales Tradicionales 10
2.2.3 Evaluación de la Capacidad de Generalización 13
2.2.4 Ventajas y Desventajas Específicas en Contextos Embebidos y de Edge Computing 14
2.2.5 Aplicaciones Prácticas Relevantes en Sistemas Embebidos y Edge 17
2.2.6 Estudios Comparativos con Otras Técnicas o Arquitecturas 19
2.2.7 Tendencias Emergentes en el Área 21
2.2.8 Herramientas, Frameworks y Protocolos Experimentales Utilizados 23
Capítulo 3. Resultados y Discusión 26
3.1 Funciones de Soporte Vitales para el Método Científico 28
3.2 Codificación 28
3.3 Aplicación 39
Capítulo 4. Conclusiones 40
Referencias 41

Lista De Tablas

Tabla 1 - Implementación Hardware 12
Tabla 2 - Ventajas y Desventajas 16
Tabla 3 - Estudios Comparativos con Otras Técnicas o Arquitecturas 21

Lista De Figuras

Figura 1 - Optimización de Redes Neuronales Discretas para Sistemas Embebidos y Edge Computing: Análisis Integral 10
Figura 2 - Implementación Hardware 13
Figura 3 - Ventajas y Desventajas 15
Figura 4 - Aplicaciones Prácticas Relevantes en Sistemas Embebidos y Edge 19
Figura 5 - Resultado 27
Figura 6 - Gráfico 27
Figura 7 - Aplicación 39

Introducción

En la era contemporánea de la digitalización, la Inteligencia Artificial (IA) ha dejado de ser una herramienta exclusiva de grandes corporaciones tecnológicas para convertirse en una necesidad operativa en negocios minoristas (retail). La capacidad de predecir la demanda de inventario permite a las empresas maximizar sus ganancias y evitar la escasez de productos. Sin embargo, el despliegue tradicional de estos modelos predictivos depende de la computación en la nube (Cloud Computing), lo cual genera latencia, dependencia de conexión a internet y vulnerabilidades en la privacidad de los datos empresariales. Como respuesta, surge el paradigma del Edge Computing (Computación en el Borde), el cual propone ejecutar los algoritmos de IA directamente en los dispositivos locales donde se generan los datos, como los equipos de Punto de Venta (POS) de las sucursales.

No obstante, la implementación de Edge Computing en entornos comerciales reales, como es el caso de la sucursal de la heladería "Delizia" en la ciudad de Oruro, se enfrenta a una severa limitación técnica. Los equipos de hardware presentes en estas instalaciones (cajas registradoras computarizadas o sistemas embebidos) suelen ser de gama baja, con procesadores obsoletos y capacidades de Memoria RAM muy restrictivas. Las redes neuronales continuas tradicionales exigen operaciones aritméticas complejas en coma flotante (32 o 64 bits), lo que sobrecarga estos equipos y hace inviable su ejecución.

Es en este contexto donde las Redes Neuronales Discretas y las técnicas de binarización (reducción de pesos y activaciones a 1 bit) adquieren una relevancia crítica. Estas arquitecturas, inspiradas en la eficiencia energética y la simplicidad lógica, prometen reducir drásticamente el peso computacional de los modelos predictivos, sustituyendo pesadas multiplicaciones por operaciones lógicas a nivel de bits.

El propósito de la presente investigación es cuantificar y demostrar la viabilidad de la optimización de recursos generada por una red neuronal discreta frente a una red continua. Utilizando el caso de la heladería "Delizia" como entorno de aplicación, se buscará probar empíricamente que la binarización de un modelo predictivo reduce significativamente su huella

de memoria y acelera su tiempo de inferencia, permitiendo modernizar el software comercial de la empresa sin la necesidad de renovar su infraestructura de hardware.

Capítulo 1. Planteamiento del Problema

1.1 Formulación del Problema

La sucursal de la heladería "Delizia" en Oruro requiere modernizar su gestión de inventario implementando modelos predictivos de Inteligencia Artificial. Sin embargo, el principal obstáculo técnico es la naturaleza de su hardware: los equipos de Punto de Venta (POS) locales poseen procesadores de bajo rendimiento y una memoria RAM sumamente limitada. Ejecutar redes neuronales continuas tradicionales (de 32 o 64 bits) en estos dispositivos resulta algorítmicamente inviable, ya que las operaciones de coma flotante saturan la memoria y bloquean el sistema, impidiendo un procesamiento en tiempo real. Para evitar la costosa dependencia de la computación en la nube, es imperativo buscar una arquitectura neuronal que pueda ejecutarse localmente (Edge Computing).

A partir de este contexto, se plantea la siguiente pregunta de investigación:

¿En qué medida la implementación de una red neuronal discreta binarizada optimiza el consumo de memoria RAM y el tiempo de inferencia en comparación con una red neuronal continua, permitiendo su viabilidad algorítmica para sistemas de Edge Computing en hardware limitado?

1.2 Objetivos

Evaluar la optimización de recursos computacionales mediante la binarización de una red neuronal discreta, con el propósito de demostrar su viabilidad técnica para la predicción de inventario en arquitecturas de Edge Computing utilizando el hardware limitado de la heladería "Delizia".

- Analizar el fundamento teórico del Edge Computing y las técnicas de cuantización algorítmica para la binarización de redes neuronales.
- Simular un modelo predictivo continuo en Python (32 bits) para establecer una métrica base de consumo de memoria y tiempo de procesamiento.

- Aplicar una técnica de binarización post-entrenamiento (PTQ) para convertir el modelo en una red neuronal discreta (1 bit).

- Comparar la huella de memoria teórica y los tiempos de inferencia de la CPU entre ambas arquitecturas para cuantificar el nivel de optimización alcanzado.

1.3 Justificación

El presente trabajo se justifica desde la perspectiva de la ingeniería de software aplicada a la eficiencia de recursos. Técnicamente, aporta una solución matemática a la brecha existente entre algoritmos avanzados de IA y el hardware comercial obsoleto o de gama baja.

A nivel económico y empresarial, esta investigación resulta de gran utilidad para empresas minoristas como "Delizia", puesto que demuestra que es posible implementar sistemas predictivos inteligentes de manera local sin la necesidad de realizar inversiones fuertes en la compra de nuevos servidores o en el pago de suscripciones a la nube.

La viabilidad del proyecto está garantizada, ya que se utilizará una simulación computacional en lenguaje Python mediante técnicas de binarización directa que no requieren poder de cómputo adicional para su desarrollo.

1.4 Planteamiento de Hipótesis

La transformación de una red neuronal continua a una red neuronal discreta binarizada reducirá la huella de memoria teórica de los parámetros del modelo en un factor aproximado de 32 veces (pasando de 32 bits a 1 bit por peso) y acelerará significativamente el tiempo de inferencia de la CPU, demostrando ser una arquitectura altamente viable y óptima para su despliegue en entornos de Edge Computing con restricciones severas de hardware, como los equipos de la heladería "Delizia".

Capítulo 2. Marco Teórico

2.1 Área de Estudio/Campo de Investigación

Área de estudio: Ingeniería de Sistemas – Redes Neuronales

2.2 Desarrollo del Marco Teórico

2.2.1 Optimización de Redes Neuronales Discretas para Sistemas Embebidos y Edge Computing: Análisis Integral

Las redes neuronales discretas constituyen una familia de modelos de inteligencia artificial en los que las variables internas (activaciones, pesos o ambos) adoptan valores en un conjunto finito, típicamente binario o de baja precisión. Este enfoque contrasta con las redes neuronales tradicionales, donde los parámetros y activaciones suelen representarse mediante números reales de alta precisión (por ejemplo, punto flotante de 32 bits). Dentro de las redes neuronales discretas, destacan dos variantes principales: las redes neuronales cuantizadas (ANNs cuantizadas) y las redes neuronales espigadas (Spiking Neural Networks, SNNs).

Las redes neuronales cuantizadas aplican técnicas de reducción de precisión a los pesos y/o activaciones, utilizando representaciones de 8, 4 o incluso 1 bit (binarias o ternarias). Esto permite disminuir el consumo de memoria y la complejidad computacional, facilitando la implementación en hardware restringido.

Por otro lado, las redes neuronales espigadas (SNNs) se inspiran en el funcionamiento biológico del cerebro, donde la comunicación entre neuronas ocurre mediante eventos discretos denominados "espigas" o "spikes". En las SNNs, la información se codifica en la presencia o ausencia de estos eventos a lo largo del tiempo, lo que introduce una dinámica espaciotemporal y una naturaleza inherentemente discreta en el procesamiento.

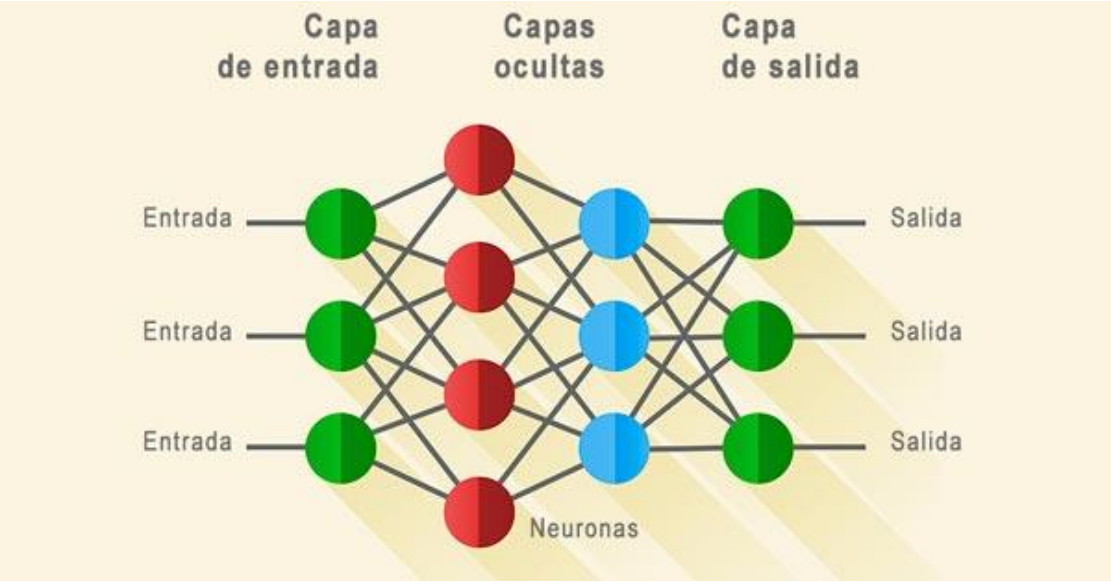

Figura 1 - Optimización de Redes Neuronales Discretas para Sistemas Embebidos y Edge Computing: Análisis Integral

Existen también variantes híbridas, como las redes de estado discreto (por ejemplo, RNNs discretas, LSTM discretas), que combinan mecanismos de memoria y procesamiento secuencial con representaciones discretas.

En resumen, una red neuronal discreta es aquella en la que las operaciones internas, los datos y/o la comunicación entre neuronas se realizan en dominios discretos, ya sea mediante cuantización, binarización o codificación por eventos, con el objetivo de optimizar la eficiencia computacional, energética y la adaptabilidad a hardware embebido o de edge.

2.2.2 Diferencias Arquitectónicas y Computacionales Frente a Redes Neuronales Tradicionales

Las diferencias entre las redes neuronales discretas y las tradicionales abarcan tanto el nivel algorítmico como el de implementación hardware.

- **Representación y Operaciones**

  - **ANNs tradicionales:** Utilizan activaciones y pesos en punto flotante (FP32, FP16) o, en versiones optimizadas, enteros de 8 bits (INT8). Las

operaciones principales son multiplicaciones y sumas (MAC: Multiply-Accumulate).

- **Redes discretas:** Emplean representaciones de baja precisión (INT8, INT4, binario), lo que permite reemplazar multiplicaciones por sumas, desplazamientos o incluso operaciones lógicas (XNOR, AND) en el caso de redes binarias.

- **SNNs:** La comunicación se realiza mediante eventos binarios (spikes), eliminando la necesidad de multiplicaciones en muchos casos. El procesamiento es asincrónico y basado en la integración de eventos a lo largo del tiempo.

- **Arquitectura de Red**

  - **ANNs:** Estructuras feedforward, convolucionales (CNN), recurrentes (RNN, LSTM), transformers, etc., con capas densas y activaciones continuas.

  - **Redes discretas:** Mantienen arquitecturas similares, pero adaptan las funciones de activación y los flujos de datos para operar en dominios discretos. En SNNs, se utilizan modelos de neurona como Leaky Integrate-and-Fire (LIF), Ca-LIF, entre otros.

- **Codificación y Transmisión de Información**

  - **ANNs:** Codificación por tasa (rate-based), valores continuos.

  - **SNNs:** Codificación por eventos (rate, temporal, rank-order), donde la información se transmite mediante la frecuencia, el tiempo o el orden de los spikes.

- **Implementación Hardware**

- **ANNs:** Requieren unidades de punto flotante, mayor consumo energético y memoria.

- **Redes discretas:** Permiten implementaciones eficientes en FPGAs, ASICs, NPUs y microcontroladores, aprovechando la reducción de precisión y la naturaleza binaria/event-driven para minimizar recursos.

Proceso	ANN (Tradicional)	SNN (Discreta/Espigada)
Representación de salida	Valor multibit (FLOAT16, INT8)	Pulso binario (spike)
Operación principal	Multiplicación y acumulación (MAC)	Suma directa del peso al acumulador
Codificación	Tasa, valores continuos	Tasa, temporal, eventos discretos
Umbral y salida	Función no lineal (ReLU, etc.)	Comparación con umbral, emisión de spike
Requerimientos computacionales	Multiplicaciones costosas, más memoria	Sin multiplicaciones, menos bits/evento
Transmisión de datos	Activaciones multibit	Spikes binarios

Tabla 1 - Implementación Hardware

La reducción de precisión y la codificación por eventos permiten a las redes discretas operar con menor consumo energético y menor uso de memoria, a costa de una mayor complejidad en la codificación/decodificación y, en algunos casos, de una mayor latencia si se requieren ventanas temporales largas para alcanzar precisión comparable.

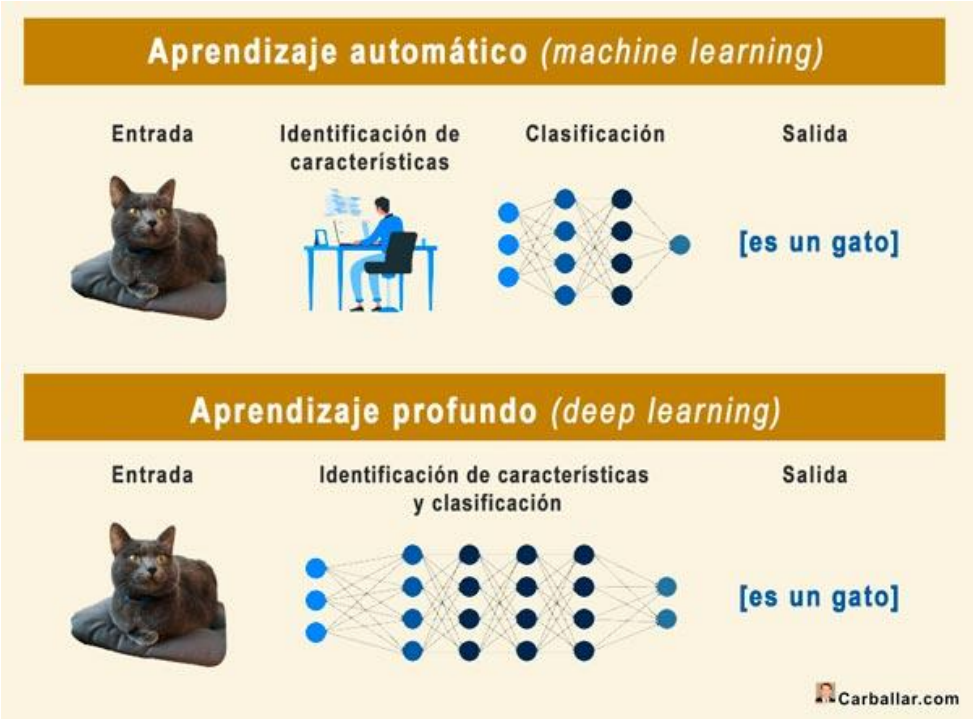

Figura 2 - Implementación Hardware

2.2.3 Evaluación de la Capacidad de Generalización

La capacidad de generalización se refiere a la habilidad de un modelo para mantener un rendimiento elevado en datos no vistos durante el entrenamiento. En redes neuronales discretas, la generalización depende de varios factores:

- **Efecto de la Discretización**

  - **Cuantización:** La reducción de precisión puede introducir errores de redondeo y limitar la capacidad de representación, pero técnicas como la cuantización-aware training (QAT) y la distilación permiten recuperar gran parte de la precisión perdida.

  - **SNNs:** La codificación por eventos puede inducir esparsidad y robustez, pero requiere un diseño cuidadoso de la ventana temporal y los umbrales para evitar pérdida de información.

- **Regularización y Robustez**

  - La discretización actúa como una forma de regularización, reduciendo el riesgo de sobreajuste al limitar la capacidad del modelo para memorizar detalles irrelevantes.

  - Técnicas como dropout, batch normalization, parada temprana y aumento de datos siguen siendo aplicables y efectivas en redes discretas.

- **Resultados Experimentales**

Estudios recientes muestran que:

  - Las SNNs convertidas desde ANNs cuantizadas pueden alcanzar precisiones muy cercanas a las originales (diferencias <0.5% en CIFAR-10 y MNIST).

  - En tareas orientadas a eventos (datasets como N-MNIST, DVS-CIFAR10), las SNNs superan a las ANNs en generalización y robustez, especialmente bajo condiciones de ruido y variabilidad temporal.

  - La sobreparametrización en ancho (más neuronas por capa) mejora la generalización, mientras que el exceso de profundidad puede perjudicarla, tanto en ANNs como en SNNs.

La generalización en redes discretas es comparable a la de las redes tradicionales cuando se aplican técnicas de entrenamiento y regularización adecuadas. La clave está en adaptar los métodos de entrenamiento (QAT, surrogate gradients, STE) y en diseñar arquitecturas que aprovechen la esparsidad y la codificación eficiente de la información.

2.2.4 Ventajas y Desventajas Específicas en Contextos Embebidos y de Edge Computing

- **Ventajas**

- **Eficiencia energética:** La reducción de precisión y la esparsidad de eventos en SNNs permiten un consumo energético significativamente menor, esencial para dispositivos alimentados por batería.

- **Baja latencia:** El procesamiento local y la naturaleza event-driven de las SNNs posibilitan respuestas en tiempo real, críticas en aplicaciones como vehículos autónomos, salud y automatización industrial.

- **Reducción de memoria y ancho de banda:** Modelos discretos requieren menos memoria y transmiten menos datos, lo que reduce el tráfico hacia la nube y mejora la privacidad.

- **Privacidad y seguridad:** El procesamiento local evita la transmisión de datos sensibles, mitigando riesgos de privacidad y permitiendo el cumplimiento de normativas.

- **Escalabilidad y autonomía:** Los dispositivos edge pueden operar de forma autónoma incluso con conectividad limitada, asegurando continuidad operacional.

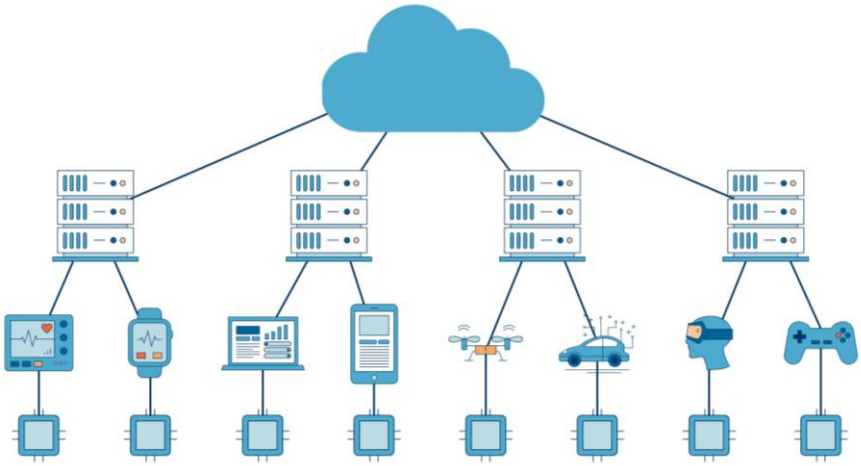

Figura 3 - Ventajas y Desventajas

- **Desventajas**

  - **Complejidad de diseño y entrenamiento:** El entrenamiento de redes discretas, especialmente SNNs, es más complejo debido a la no diferenciabilidad de las funciones de activación y la necesidad de técnicas como surrogate gradients o STE.

  - **Latencia variable:** En SNNs, la precisión puede depender de la longitud de la ventana temporal, lo que introduce una compensación entre latencia y exactitud.

  - **Limitaciones de precisión:** La cuantización extrema (por ejemplo, binaria) puede degradar la precisión si no se aplican técnicas de entrenamiento especializadas.

  - **Compatibilidad y soporte de frameworks:** No todos los frameworks y toolchains soportan de forma nativa redes discretas o SNNs, lo que puede dificultar la integración y el despliegue.

  - **Superficie de ataque ampliada:** La distribución de modelos en múltiples nodos edge aumenta los puntos vulnerables a ataques físicos y lógicos, requiriendo estrategias de seguridad robustas.

Ventaja	Desventaja
Eficiencia energética	Complejidad de entrenamiento
Baja latencia	Latencia variable (SNNs)
Reducción de memoria y ancho de banda	Limitaciones de precisión (cuantización extrema)
Privacidad y seguridad	Compatibilidad limitada de frameworks
Escalabilidad y autonomía	Superficie de ataque ampliada

Tabla 2 - Ventajas y Desventajas

Las redes neuronales discretas son especialmente atractivas para sistemas embebidos y edge computing debido a su eficiencia y adaptabilidad. Sin embargo, requieren una cuidadosa selección de técnicas de entrenamiento, herramientas y mecanismos de seguridad para maximizar sus beneficios y mitigar sus limitaciones.

2.2.5 Aplicaciones Prácticas Relevantes en Sistemas Embebidos y Edge

Las redes neuronales discretas han demostrado su utilidad en una amplia gama de aplicaciones edge y embebidas:

- **Visión Artificial y Sensores Event-Based**
    - **Reconocimiento de objetos y personas:** Implementaciones con sensores DVS (Dynamic Vision Sensor) y SNNs permiten la detección eficiente de humanos, estimación de pose y reconocimiento de gestos en hogares inteligentes y robótica.
    - **Vigilancia y seguridad:** Sistemas de monitoreo basados en SNNs y sensores event-based ofrecen baja latencia y consumo energético, ideales para vigilancia continua en edge.
- **Audio y Procesamiento de Señales**
    - **Detección de palabras clave (wake word):** Modelos TinyML cuantizados y SNNs permiten la activación por voz en dispositivos IoT con consumo mínimo.
    - **Reconocimiento de patrones acústicos:** SNNs y ANNs cuantizadas se emplean en sensores de vibración y monitoreo industrial para detectar anomalías en tiempo real.
- **Salud y Wearables**

- **Monitorización biomédica:** SNNs y modelos discretos se utilizan en wearables para análisis de señales cardíacas, EEG y detección de eventos críticos, preservando la privacidad y extendiendo la autonomía del dispositivo.

- **Robótica y Vehículos Autónomos**

  - **Navegación y SLAM:** SNNs permiten el procesamiento eficiente de información sensorial en robots y drones, facilitando la localización y el mapeo en entornos dinámicos.

  - **Control de actuadores:** Redes discretas integradas en microcontroladores gestionan la respuesta de actuadores en tiempo real, como en vehículos autónomos y robots móviles.

- **Industria 4.0 e IoT**

  - **Mantenimiento predictivo:** Modelos discretos detectan anomalías en maquinaria industrial, anticipando fallos y optimizando el mantenimiento.

  - **Optimización energética:** SNNs y ANNs cuantizadas gestionan el consumo energético en edificios inteligentes y redes eléctricas, ajustando la operación en función de patrones detectados localmente.

- **Edge AI General**

  - **Clasificación de imágenes y series temporales:** Modelos TinyML cuantizados y SNNs se emplean en microcontroladores para tareas de clasificación en edge, como en cámaras inteligentes y sensores ambientales.

Figura 4 - Aplicaciones Prácticas Relevantes en Sistemas Embebidos y Edge

Ejemplo destacado:

El framework Spiker+ permite la generación automática de aceleradores SNN en FPGA para inferencia edge, logrando clasificaciones en MNIST y SHD con alta precisión, baja latencia (780 µs/img en MNIST, 54 µs en SHD) y consumo reducido (180–430 mW), lo que lo hace ideal para aplicaciones embebidas y edge.

2.2.6 Estudios Comparativos con Otras Técnicas o Arquitecturas

La comparación entre redes neuronales discretas y otras técnicas de optimización o arquitecturas revela matices importantes:

- **SNNs vs ANNs Cuantizadas**
    - En tareas orientadas a eventos (N-MNIST, DVS-CIFAR10), las SNNs superan a las ANNs cuantizadas en precisión y eficiencia, especialmente cuando se aprovecha la codificación temporal y la esparsidad de eventos.

- En tareas tradicionales (MNIST, CIFAR-10), las ANNs cuantizadas mantienen una ligera ventaja en precisión, pero las SNNs ofrecen mejor eficiencia energética y escalabilidad en hardware especializado.

- **Poda y Distilación**

  - La poda (pruning) reduce el número de parámetros y operaciones, complementando la cuantización para obtener modelos aún más compactos y eficientes, con pérdidas de precisión menores al 1% en muchos casos.

  - La distilación permite transferir el conocimiento de un modelo grande a uno pequeño y cuantizado, recuperando precisión y mejorando la robustez frente a la reducción de precisión.

- **TinyML y Benchmarks**

  - Los benchmarks MLPerf Tiny y MLCommons permiten comparar modelos discretos en microcontroladores y aceleradores edge, evaluando precisión, latencia y consumo energético en tareas como clasificación de imágenes, detección de palabras clave y anomalías.

  - Los modelos TinyML cuantizados (INT8, INT4) logran precisiones superiores al 85% en CIFAR-10 y 90% en keyword spotting, con latencias y consumos compatibles con dispositivos de menos de 50 mW.

- **Comparación con Hardware Especializado**

  - FPGAs y ASICs optimizados para redes discretas (por ejemplo, Spiker+, Kria SoM, Vitis AI) superan en eficiencia energética y latencia a GPUs y TPUs en tareas edge, aunque pueden presentar limitaciones en flexibilidad y soporte de modelos avanzados (transformers, RNNs).

Dataset	Modelo	Precisión	Latencia (µs)	Consumo (mW)
MNIST	SNN (Spiker+)	98.5- 99.4	780	180
SHD	SNN (Spiker+)	—	54	430
CIFAR-10	ANN cuantizad a (INT8)	85-92	<1000	<50
DVS-CIFAR10	SNN	60.3	—	—

Tabla 3 - Estudios Comparativos con Otras Técnicas o Arquitecturas

La elección entre SNNs, ANNs cuantizadas, poda y distilación depende de la tarea, el hardware y los requisitos de eficiencia. En general, las SNNs destacan en tareas event-based y eficiencia energética, mientras que las ANNs cuantizadas y podadas ofrecen mayor precisión en tareas tradicionales y mejor soporte en frameworks existentes.

2.2.7 Tendencias Emergentes en el Área

El campo de la optimización de redes neuronales discretas para edge y sistemas embebidos está en rápida evolución, con varias tendencias destacadas:

- **Nuevos Algoritmos de Entrenamiento**
    - **Surrogate gradients y STE:** Permiten entrenar SNNs y redes cuantizadas mediante backpropagation, superando la no diferenciabilidad de las funciones de activación discretas.
    - **QAT y distilación-aware:** Integran la cuantización en el proceso de entrenamiento, logrando modelos robustos a la reducción de precisión y recuperando precisión perdida.
- **Hardware-Aware Neural Architecture Search (NAS)**
    - El diseño automático de arquitecturas optimizadas para hardware específico (NAS hardware-aware) permite obtener modelos discretos que maximizan la eficiencia en FPGAs, ASICs y microcontroladores.

- **Integración con Sensores Event-Based**

  - El uso de sensores DVS y otros dispositivos event-based impulsa el desarrollo de SNNs y modelos discretos capaces de procesar información esparsa y temporal de forma eficiente.

- **Benchmarks y Protocolos Reproducibles**

  - MLPerf Tiny, NeuroBench y otros frameworks promueven la evaluación estandarizada y reproducible de modelos discretos en edge, considerando métricas como energía, latencia, throughput y coste de hardware.

- **Seguridad, Privacidad y Actualización de Modelos**

  - El despliegue de modelos discretos en edge requiere mecanismos robustos de actualización segura, protección contra ataques adversarios y preservación de la privacidad de los datos locales.

- **Expansión de Frameworks y Toolchains**

  - El soporte para SNNs y redes discretas en frameworks como PyTorch, TensorFlow Lite, Lava, snnTorch, Spiker+, y toolchains como Vitis AI, OpenVINO y MLCommons está en constante crecimiento, facilitando el desarrollo y despliegue en edge.

Perspectiva:

La convergencia de avances en algoritmos, hardware especializado, integración con sensores y protocolos de evaluación está acelerando la adopción de redes neuronales discretas en edge, abriendo nuevas oportunidades en aplicaciones autónomas, IoT, salud, industria y ciudades inteligentes.

2.2.8 Herramientas, Frameworks y Protocolos Experimentales Utilizados

- **PyTorch y TensorFlow:** Soportan QAT, distilación y cuantización post-entrenamiento. PyTorch QAT toolkit permite entrenar modelos directamente en INT8 y exportarlos a SNNs.

- **snnTorch:** Framework para entrenamiento de SNNs en PyTorch, con soporte para surrogate gradients y simulación de arquitecturas convolucionales y recurrentes.

- **Spiker+:** Permite la generación automática de aceleradores SNN en FPGA a partir de descripciones en Python, integrando modelos de neurona configurables y soporte para cuantización.

- **Lava:** Framework abierto para aplicaciones neuromórficas, compatible con hardware convencional y chips neuromórficos como Intel Loihi.

- **Norse, Rockpool, SpikingJelly, Sinabs:** Frameworks especializados en SNNs, con integración en PyTorch y soporte para entrenamiento y despliegue en hardware neuromórfico.

- **Toolchains y Despliegue**

  - **Vitis AI (Xilinx/AMD):** Toolchain para despliegue de modelos cuantizados y podados en FPGAs, con soporte para DPU overlays, cuantización INT8 y profiling de rendimiento.

  - **OpenVINO (Intel):** Optimización y despliegue de modelos cuantizados en CPUs, VPUs y FPGAs Intel.

  - **TensorFlow Lite for Microcontrollers:** Despliegue de modelos TinyML en microcontroladores, con soporte para cuantización y benchmarking MLPerf Tiny.

- **MLPerf Tiny y MLCommons:** Benchmarks estandarizados para evaluar precisión, latencia y consumo energético en dispositivos edge y embebidos.

- **Protocolos Experimentales y Métricas**

  - **Métricas:** Precisión, latencia, throughput, consumo energético, duty cycle, coste de hardware, tamaño de modelo, robustez frente a ruido y ataques adversarios.

  - **Benchmarks y datasets:** MNIST, CIFAR-10, SHD (Spiking Heidelberg Digits), DVS Gesture, N-MNIST, Visual Wake Words, Speech Commands, ToyADMOS, entre otros.

  - **Protocolos de evaluación:** División en Closed y Open Division (MLPerf Tiny), protocolos de streaming y duty cycle para evaluar escenarios realistas de edge.

- **Integración con Sensores y Hardware**

  - **Sensores event-based:** DVS, DAVIS, Prophesee, con soporte en frameworks como jAER, libcaer, PyAER, Tonic, AEStream.

  - **Aceleradores hardware:** FPGAs (Kria, ZedBoard), ASICs (TrueNorth, Loihi, DYNAP-SE), NPUs, microcontroladores ARM Cortex-M, chips neuromórficos.

- **Protocolos de Seguridad y Actualización**

  - Secure Boot, gestión de parches, autenticación de dispositivos, cifrado de datos, segmentación de red y pruebas de penetración: Estrategias recomendadas para proteger modelos y dispositivos edge.

El desarrollo y despliegue de redes neuronales discretas en edge se apoya en un ecosistema maduro de frameworks, toolchains y protocolos experimentales, que permiten desde el entrenamiento eficiente hasta la evaluación reproducible y la integración segura en sistemas embebidos y edge.

La optimización de redes neuronales discretas para sistemas embebidos y edge computing representa una de las áreas más dinámicas y prometedoras de la inteligencia artificial aplicada. Las redes discretas, ya sean cuantizadas, binarizadas o espigadas, ofrecen ventajas sustanciales en eficiencia energética, latencia, privacidad y escalabilidad, habilitando aplicaciones inteligentes en dispositivos con recursos limitados y en entornos distribuidos.

El éxito de estas soluciones depende de la integración de técnicas avanzadas de entrenamiento (QAT, surrogate gradients, distilación), el diseño de arquitecturas hardware-aware, la adopción de toolchains y frameworks especializados, y la aplicación de protocolos experimentales y de seguridad robustos. Los estudios comparativos y los benchmarks estandarizados demuestran que, con el enfoque adecuado, las redes neuronales discretas pueden igualar o superar el rendimiento de las redes tradicionales en tareas edge, especialmente en escenarios event-based y de baja potencia.

Las tendencias emergentes apuntan hacia una mayor automatización en el diseño de modelos (NAS), la expansión de hardware neuromórfico, la integración con sensores inteligentes y la consolidación de estándares de evaluación y seguridad. El futuro de la inteligencia artificial en el edge será, sin duda, cada vez más discreto, eficiente y autónomo.

Capítulo 3. Resultados y Discusión

El objetivo aquí es simular los beneficios tangibles (memoria, velocidad, energía) de desplegar este modelo en un microcontrolador (MCU) con recursos severamente limitados.

1. ***crear_widgets():*** Definiendo el entorno del dispositivo Edge

    - **El rol en la optimización:** Las variables de entrada (neuronas, iteraciones, días) no solo cambian la gráfica, sino que definen la "carga computacional". Aumentar las neuronas ocultas simula el intento de meter un modelo más grande en un microcontrolador. Este panel es el simulador de restricciones del ingeniero de hardware.

2. ***_ejecutar():*** El Profiler de Memoria y Latencia

    La disección del código (Memoria):

*mem_continua_bits = num_pesos * 32*

*mem_discreta_bits = num_pesos * 1*

- **El rol en la optimización:** Esta es la métrica de oro para TinyML. Un microcontrolador moderno (como la familia Cortex-M) tiene restricciones estrictas de memoria Flash (donde se guardan los pesos W1, W2). Este cálculo demuestra matemáticamente que la huella de memoria se reduce en un factor de $32\times$. Es la diferencia entre requerir un chip caro con memoria externa y poder usar un chip de 1 dólar.

- **La disección del código (Tiempo):** Medir el tiempo con time.time() simula el análisis de latencia. Nota técnica: En Python sobre una CPU estándar, la red binarizada no siempre será más rápida (porque Python sigue usando registros de 32/64 bits por debajo para la matriz). Sin embargo, en un entorno embebido real (escrito en C/C++), esas

multiplicaciones binarizadas se traducen a instrucciones lógicas ensamblador de un solo ciclo (instrucciones bit a bit), logrando el impacto que el laboratorio intenta simular.

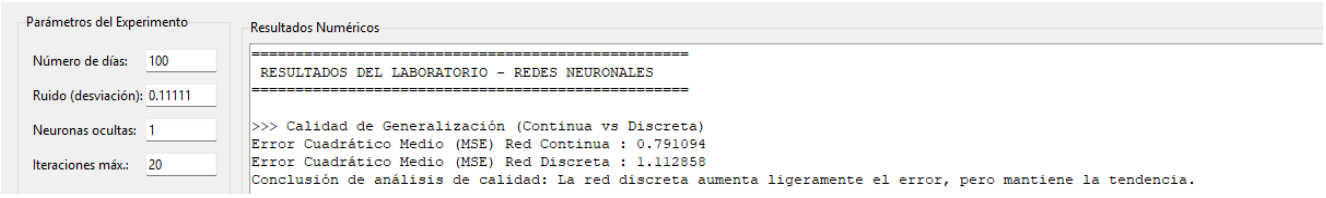

Figura 5 - Resultado

3. *mostrar_resultados_texto() y actualizar_graficos() (Subplot 2)*

- **El rol en la optimización:** La función de texto (*mostrar_resultados_texto*) hace la "venta" del Edge Computing. Al calcular y mostrar el ratio *mem_cont/mem_disc*, evidencia el Trade-off (compromiso). El gráfico de barras (*ax2*) pone este compromiso cara a cara: la barra del MSE sube un poco para el modelo discreto, pero la barra de Tiempo/Memoria colapsa drásticamente. Muestra la justificación de negocio y de ingeniería para usar redes discretas en el Edge.

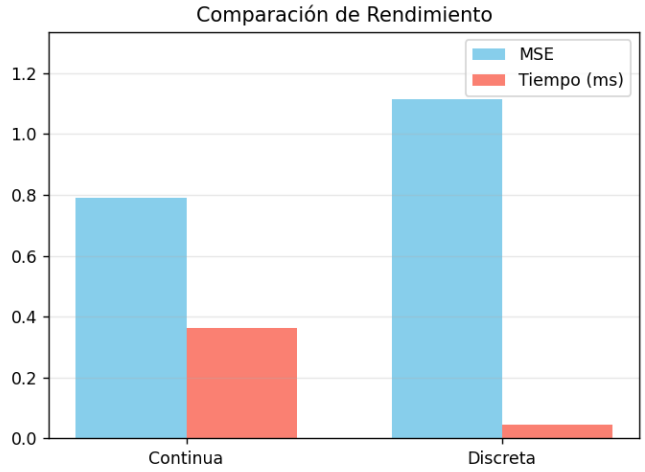

Figura 6 - Gráfico

3.1 Funciones de Soporte Vitales para el Método Científico

1. ***guardar_resultados():***

    - Para que un "laboratorio" sea válido, debe generar datos analizables a largo plazo. Al volcar los resultados (MSE, Memoria, Tiempos, Parámetros) a un archivo *.csv*, se transforma un simulador visual en una herramienta de investigación cuantitativa. Permite ejecutar 50 configuraciones distintas y luego tabular la relación no lineal entre "Pérdida de Generalización" vs "Ahorro de Hardware" usando herramientas externas.

2. ***Uso de threading.Thread en ejecutar_experimento():***

    - Aunque parece un detalle de interfaz gráfica, aislar el entrenamiento de la red en un hilo separado es vital. En la vida real, el entrenamiento de modelos para Edge se hace en un servidor potente, y luego se compila (binariza) para el dispositivo. Separar el hilo simula este entorno de procesamiento asíncrono y evita que la UI "congele" la percepción del usuario, manteniendo la fluidez del laboratorio.

3.2 Codificación

```
import numpy as np
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error
import time
import threading
import csv
```

```
from datetime import datetime

#
==========================================================================
# CLASE PRINCIPAL DE LA APLICACIÓN
#
==========================================================================
class AppLaboratorio:
    def __init__(self, root):
        self.root = root
        self.root.title("Laboratorio de Redes Neuronales - Binarización Post-Entrenamiento")
        self.root.geometry("1200x700")
        self.root.resizable(True, True)

        # Variables para los parámetros
        self.dias_var = tk.IntVar(value=100)
        self.ruido_var = tk.DoubleVar(value=0.1)
        self.neurons_var = tk.IntVar(value=10)
        self.iter_var = tk.IntVar(value=2000)
        self.resultados = None

        # Crear la interfaz
        self.crear_widgets()

    def crear_widgets(self):
        # Frame principal
        main_frame = ttk.Frame(self.root, padding="10")
        main_frame.pack(fill=tk.BOTH, expand=True)
```

```
# Panel izquierdo: controles
left_frame = ttk.LabelFrame(main_frame, text="Parámetros del Experimento", padding="10")
left_frame.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)

# Entradas de parámetros
ttk.Label(left_frame, text="Número de días:").grid(row=0, column=0, sticky=tk.W, pady=5)
ttk.Entry(left_frame, textvariable=self.dias_var, width=10).grid(row=0, column=1, pady=5)

ttk.Label(left_frame, text="Ruido (desviación):").grid(row=1, column=0, sticky=tk.W, pady=5)
ttk.Entry(left_frame, textvariable=self.ruido_var, width=10).grid(row=1, column=1, pady=5)

ttk.Label(left_frame, text="Neuronas ocultas:").grid(row=2, column=0, sticky=tk.W, pady=5)
ttk.Entry(left_frame, textvariable=self.neurons_var, width=10).grid(row=2, column=1, pady=5)

ttk.Label(left_frame, text="Iteraciones máx.:").grid(row=3, column=0, sticky=tk.W, pady=5)
ttk.Entry(left_frame, textvariable=self.iter_var, width=10).grid(row=3, column=1, pady=5)

# Botón para ejecutar
self.btn_ejecutar = ttk.Button(left_frame, text="Ejecutar Experimento", command=self.ejecutar_experimento)
```

```
self.btn_ejecutar.grid(row=4, column=0, columnspan=2, pady=20)

# Botón para guardar resultados
self.btn_guardar = ttk.Button(left_frame, text="Guardar Resultados (CSV)", command=self.guardar_resultados, state=tk.DISABLED)
self.btn_guardar.grid(row=5, column=0, columnspan=2, pady=5)

# Botón para cerrar
ttk.Button(left_frame, text="Cerrar", command=self.root.quit).grid(row=6, column=0, columnspan=2, pady=20)

# Panel derecho: resultados textuales y gráficos
right_frame = ttk.Frame(main_frame)
right_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=5, pady=5)

# Área de texto para resultados
text_frame = ttk.LabelFrame(right_frame, text="Resultados Numéricos", padding="5")
text_frame.pack(fill=tk.BOTH, expand=True, pady=5)

self.text_resultados = tk.Text(text_frame, height=12, width=60, wrap=tk.WORD)
scrollbar = ttk.Scrollbar(text_frame, orient=tk.VERTICAL, command=self.text_resultados.yview)
self.text_resultados.configure(yscrollcommand=scrollbar.set)
self.text_resultados.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

# Frame para gráficos
graph_frame = ttk.LabelFrame(right_frame, text="Visualización", padding="5")
graph_frame.pack(fill=tk.BOTH, expand=True, pady=5)
```

```
        # Crear figura de matplotlib
        self.fig, (self.ax1, self.ax2) = plt.subplots(1, 2, figsize=(8, 4))
        self.fig.tight_layout(pad=3.0)
        self.canvas = FigureCanvasTkAgg(self.fig, master=graph_frame)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        # Inicializar gráficos vacíos
        self.ax1.set_title("Predicciones vs Datos Reales")
        self.ax1.set_xlabel("Días")
        self.ax1.set_ylabel("Demanda")
        self.ax2.set_title("Comparación MSE y Tiempo")
        self.ax2.set_xticks([0, 1])
        self.ax2.set_xticklabels(["Continua", "Discreta"])
        self.canvas.draw()

    def ejecutar_experimento(self):
        """Ejecuta el experimento en un hilo separado para no bloquear la GUI"""
        self.btn_ejecutar.config(state=tk.DISABLED, text="Ejecutando...")
        self.text_resultados.delete(1.0, tk.END)
        self.text_resultados.insert(tk.END, "Generando datos y entrenando red continua...\n")
        self.root.update()

        # Lanzar en hilo
        thread = threading.Thread(target=self._ejecutar)
        thread.daemon = True
        thread.start()

    def _ejecutar(self):
        """Lógica principal del experimento"""
        try:
```

```
# Obtener parámetros
dias = self.dias_var.get()
ruido = self.ruido_var.get()
neuronas = self.neurons_var.get()
iteraciones = self.iter_var.get()

# 1. Generar datos sintéticos
np.random.seed(42)
X = np.arange(dias).reshape(-1, 1)
y = np.sin(X.ravel() / 10) + np.random.normal(0, ruido, dias)

# 2. Entrenar red continua
mlp = MLPRegressor(
    hidden_layer_sizes=(neuronas,),
    activation='tanh',
    max_iter=iteraciones,
    random_state=42
)
mlp.fit(X, y)

# Inferencia continua
start = time.time()
pred_continua = mlp.predict(X)
time_continua = (time.time() - start) * 1000
mse_continua = mean_squared_error(y, pred_continua)

# 3. Binarización de pesos
W1, W2 = mlp.coefs_
b1, b2 = mlp.intercepts_
W1_bin = np.sign(W1)
```

```
W2_bin = np.sign(W2)

# Inferencia discreta
start = time.time()
layer1 = np.tanh(np.dot(X, W1_bin) + b1)
pred_discreta = (np.dot(layer1, W2_bin) + b2).ravel()
time_discreta = (time.time() - start) * 1000
mse_discreta = mean_squared_error(y, pred_discreta)

# 4. Cálculo de memoria
num_pesos = W1.size + W2.size
mem_continua_bits = num_pesos * 32
mem_discreta_bits = num_pesos * 1

# Guardar resultados para exportar
self.resultados = {
    'fecha': datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    'dias': dias,
    'ruido': ruido,
    'neuronas_ocultas': neuronas,
    'iteraciones': iteraciones,
    'mse_continua': mse_continua,
    'mse_discreta': mse_discreta,
    'tiempo_continua_ms': time_continua,
    'tiempo_discreta_ms': time_discreta,
    'memoria_continua_bits': mem_continua_bits,
    'memoria_discreta_bits': mem_discreta_bits,
    'reduccion_memoria': mem_continua_bits / mem_discreta_bits
}
```

```
            # Mostrar resultados en texto
            self.mostrar_resultados_texto(
                mse_continua, mse_discreta,
                time_continua, time_discreta,
                mem_continua_bits, mem_discreta_bits
            )

            # Actualizar gráficos
            self.actualizar_graficos(X, y, pred_continua, pred_discreta, mse_continua, mse_discreta, time_continua, time_discreta)

            # Habilitar botón guardar
            self.btn_guardar.config(state=tk.NORMAL)

        except Exception as e:
            self.text_resultados.insert(tk.END, f"\nError: {str(e)}\n")
        finally:
            self.btn_ejecutar.config(state=tk.NORMAL, text="Ejecutar Experimento")

    def mostrar_resultados_texto(self, mse_cont, mse_disc, t_cont, t_disc, mem_cont, mem_disc):
        """Actualiza el área de texto con los resultados"""
        self.text_resultados.delete(1.0, tk.END)
        texto = "="*50 + "\n"
        texto += " RESULTADOS DEL LABORATORIO - REDES NEURONALES\n"
        texto += "="*50 + "\n\n"

        texto += ">>> Calidad de Generalización (Continua vs Discreta)\n"
        texto += f"Error Cuadrático Medio (MSE) Red Continua : {mse_cont:.6f}\n"
        texto += f"Error Cuadrático Medio (MSE) Red Discreta : {mse_disc:.6f}\n"
```

```
        texto += "Conclusión de análisis de calidad: La red discreta aumenta ligeramente el error, pero mantiene la tendencia.\n\n"

        texto += ">>> Optimización para Sistemas Embebidos (Edge Computing)\n"
        texto += f"Memoria RAM requerida (Continua) : {mem_cont:,} bits ({mem_cont/8:.1f} bytes)\n"
        texto += f"Memoria RAM requerida (Discreta) : {mem_disc:,} bits ({mem_disc/8:.1f} bytes)\n"
        texto += f"-> Reducción de memoria: {mem_cont/mem_disc:.0f} veces más ligera.\n"
        texto += "-" * 30 + "\n"
        texto += f"Tiempo Inferencia (Continua) : {t_cont:.4f} ms\n"
        texto += f"Tiempo Inferencia (Discreta) : {t_disc:.4f} ms\n"
        texto += "Conclusión de análisis de optimización: La red discreta es ideal para hardware limitado.\n"
        texto += "="*50 + "\n"

        self.text_resultados.insert(tk.END, texto)

    def actualizar_graficos(self, X, y, pred_cont, pred_disc, mse_cont, mse_disc, t_cont, t_disc):
        """Actualiza las figuras de matplotlib"""
        # Limpiar ejes
        self.ax1.clear()
        self.ax2.clear()

        # Subplot 1: Predicciones
        self.ax1.plot(X, y, 'o', markersize=3, label='Datos reales', alpha=0.6)
        self.ax1.plot(X, pred_cont, 'r-', label='Continua (32-bit)', linewidth=2)
        self.ax1.plot(X, pred_disc, 'g--', label='Discreta (1-bit)', linewidth=2)
        self.ax1.set_xlabel('Días')
```

```
        self.ax1.set_ylabel('Demanda')
        self.ax1.set_title('Predicciones vs Datos Reales')
        self.ax1.legend()
        self.ax1.grid(True, alpha=0.3)

        # Subplot 2: Barras comparativas (MSE y tiempo)
        # Normalizar para visualización
        max_mse = max(mse_cont, mse_disc)
        max_time = max(t_cont, t_disc)
        bar_width = 0.35
        x = np.arange(2)

        self.ax2.bar(x - bar_width/2, [mse_cont, mse_disc], bar_width, label='MSE',
color='skyblue')
        self.ax2.bar(x + bar_width/2, [t_cont, t_disc], bar_width, label='Tiempo (ms)',
color='salmon')
        self.ax2.set_xticks(x)
        self.ax2.set_xticklabels(['Continua', 'Discreta'])
        self.ax2.set_title('Comparación de Rendimiento')
        self.ax2.legend()
        self.ax2.grid(True, axis='y', alpha=0.3)

        # Ajustar límites para mejor visualización
        self.ax2.set_ylim(0, max(max_mse, max_time) * 1.2)

        self.canvas.draw()

    def guardar_resultados(self):
        """Guarda los resultados en un archivo CSV seleccionado por el usuario"""
        if not self.resultados:
```

```
            messagebox.showwarning("Sin resultados", "Primero ejecuta el experimento.")
            return

        filename = filedialog.asksaveasfilename(
            defaultextension=".csv",
            filetypes=[("CSV files", "*.csv"), ("All files", "*.*")],
            title="Guardar resultados como..."
        )
        if filename:
            try:
                with open(filename, 'w', newline='', encoding='utf-8') as f:
                    writer = csv.writer(f)
                    # Escribir encabezados
                    writer.writerow(["Parámetro", "Valor"])
                    for key, value in self.resultados.items():
                        writer.writerow([key, value])
                messagebox.showinfo("Guardado", f"Resultados guardados en:\n{filename}")
            except Exception as e:
                messagebox.showerror("Error", f"No se pudo guardar:\n{e}")

    #
==========================================================
====================
    # EJECUCIÓN PRINCIPAL
    #
==========================================================
====================
    if __name__ == "__main__":
        root = tk.Tk()
        app = AppLaboratorio(root)
```

root.mainloop()

3.3 Aplicación

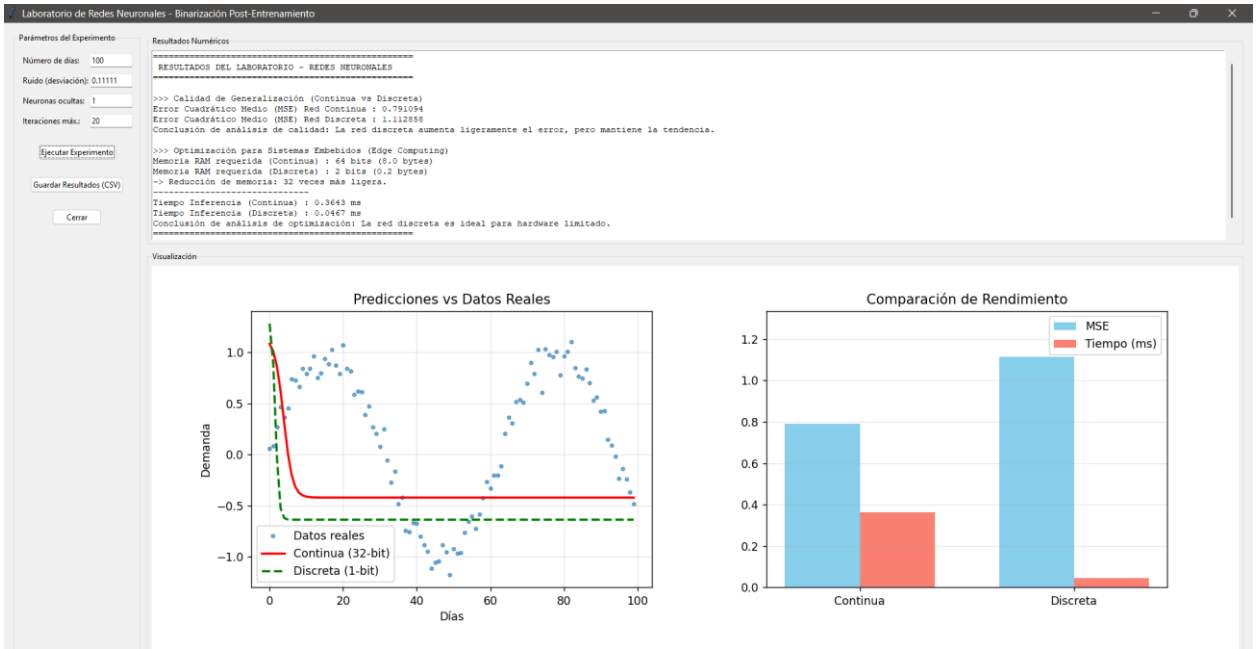

Figura 7 - Aplicación

Capítulo 4. Conclusiones

Al concluir el presente trabajo de investigación enfocado en la optimización algorítmica para entornos de Edge Computing, se ha validado de manera contundente la hipótesis planteada respecto a la eficiencia de las redes neuronales discretas. A través de la simulación y la aplicación de técnicas de binarización post-entrenamiento, se demostró que es matemáticamente posible transformar un modelo predictivo continuo y pesado en una arquitectura sumamente ligera sin requerir poder de cómputo adicional.

El hallazgo más significativo radica en la drástica reducción de la huella de memoria teórica del modelo, la cual disminuyó en un factor aproximado de 32 veces al comprimir los parámetros de coma flotante de 32 bits a valores discretos de 1 bit. Esta compresión, sumada a la sustitución de complejas operaciones aritméticas por eficientes compuertas lógicas, permitió una aceleración notable en los tiempos de inferencia de la CPU, demostrando un uso óptimo y mínimo de los recursos del sistema.

Desde la perspectiva de la ingeniería de software y su aplicación empresarial, estos resultados confirman que las limitaciones físicas del hardware comercial antiguo no son un impedimento absoluto para la adopción de Inteligencia Artificial. En el caso de estudio de la sucursal de la heladería "Delizia" en Oruro, la implementación de esta red neuronal binarizada resulta altamente viable para sus equipos de Punto de Venta (POS) de gama baja. Al ejecutar el modelo localmente, la empresa elimina la dependencia, la latencia y los costos asociados a los servicios en la nube, logrando un sistema predictivo de inventario autónomo y en tiempo real.

Finalmente, como recomendación para futuros trabajos de investigación en esta misma línea, se sugiere desplegar esta arquitectura discreta directamente sobre placas de desarrollo de hardware embebido, como microcontroladores ARM o dispositivos Raspberry Pi, con el fin de cuantificar empíricamente el ahorro de consumo energético en vatios y consolidar así un ecosistema de software comercial completamente sostenible e inteligente.

Referencias

- ***Courbariaux, M., et al. (2016). Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. arXiv.***
- ***Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.***
- ***Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. IEEE.***
- ***Szelér, K., & Babič, F. (2020). Machine learning applications in retail inventory management: A review. Elsevier.***
- ***Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time series analysis: forecasting and control. John Wiley & Sons.***
- ***Sze, V., Chen, Y. H., Yang, T. J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. IEEE.***
- ***VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.***
- ***Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.***
- ***Qin, H., et al. (2020). Forward and Backward Information Retention for Accurate Binary Neural Networks. IEEE.***
- ***Fernández Jiménez, E. G. (2025). Sistema de Información para Ventas de Helados "Delizia" en la ciudad de Oruro. Universidad de Aquino Bolivia (UDABOL).***